

Hai công cụ sắc bén cho bài toán thống kê động

Phân tích cây đoạn và cắt hình chữ nhật

Xue Mao

Nguồn gốc: Two sharp tools for dynamic statistics: segment trees and rectangle cutting

IOI China candidate team papers / 2004 / Xue Mao.pdf

Tóm tắt

Xuất phát từ các bài toán thống kê, bài viết giới thiệu hai phương pháp để xử lý chúng hiệu quả hơn: cây đoạn và cắt hình chữ nhật. Phần cây đoạn trình bày cấu trúc cơ bản, cách xây dựng, thao tác chèn và xóa đoạn, các cải tiến bằng đánh dấu lười, cũng như mở rộng lên nhiều chiều. Phần cắt hình chữ nhật trình bày ý tưởng bắt đầu từ cắt đoạn, cách cắt hình chữ nhật, mở rộng lên hình hộp và không gian nhiều chiều, rồi áp dụng vào các bài thống kê. Cuối cùng, bài viết so sánh độ phức tạp, ưu khuyết điểm và phạm vi thích hợp của hai phương pháp, từ đó đưa ra kinh nghiệm chọn công cụ trong từng tình huống.

Từ khóa. Cây đoạn; cây hình chữ nhật; cây hình hộp; cắt đoạn; cắt hình chữ nhật.

1 Dẫn nhập

Trong luyện tập và thi lập trình, ta thường gặp các bài toán kiểu thống kê. Đề bài cung cấp thông tin về một hệ thống thông qua dữ liệu vào, rồi yêu cầu chương trình trả lời hiệu quả trạng thái của hệ thống tại một thời điểm hoặc trong một tình huống nào đó. Những bài toán này thường có mô tả đơn giản và cũng dễ viết mô phỏng trực tiếp. Tuy nhiên, khi quy mô dữ liệu lớn, mô phỏng thường không đáp ứng được yêu cầu, vì vậy cần tìm phương pháp tốt hơn. Bài viết này giới thiệu hai phương pháp như vậy: cây đoạn và cắt hình chữ nhật.

2 Cây đoạn

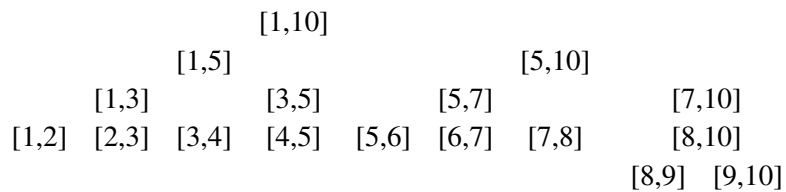
Cây đoạn đã là một khái niệm quen thuộc, nên phần này chỉ giới thiệu ngắn gọn những thao tác cần thiết cho các phần sau.

2.1 Cấu trúc của cây đoạn

Cây đoạn là một cây nhị phân. Mỗi nút của nó biểu diễn một đoạn $[a, b]$; hai con trái và phải lần lượt biểu diễn hai nửa của đoạn:

$$\left[a, \left\lfloor \frac{a+b}{2} \right\rfloor \right], \quad \left[\left\lceil \frac{a+b}{2} \right\rceil, b \right].$$

Nút lá là đoạn đơn vị $[a, a + 1]$. Ví dụ, cây đoạn có gốc $[1, 10]$ có dạng:



Dễ chứng minh cây đoạn có gốc $[a, b]$ có $2(b - a) - 1$ nút. Vì cây cân bằng, độ sâu của nó là $\log_2(2(b - a))$.

Nút cây thường được mô tả bằng cấu trúc:

```

Type TreeNode = Record
  a, b, Left, Right: Longint;
End;

```

Trong đó a, b là hai đầu nút của đoạn, còn Left, Right là chỉ số hai con. Ta có thể dùng một mảng một chiều để lưu cây:

```

Tree: array[1..Maxn] of TreeNode;

```

Bốn trường $a, b, \text{Left}, \text{Right}$ là tối thiểu. Tùy bài toán, ta có thể thêm các trường khác, chẳng hạn Cover để ghi số lần đoạn được phủ, hoặc bj để ghi dấu sửa đổi lười.

2.2 Xây dựng cây đoạn

Cây đoạn được xây bằng một thủ tục đệ quy đơn giản:

```

Procedure MakeTree(a, b)
Var Now: Longint
Begin
  tot <- tot + 1
  Now <- tot
  Tree[Now].a <- a
  Tree[Now].b <- b
  If a + 1 < b then
    Tree[Now].Left <- tot + 1
    MakeTree(a, floor((a + b) / 2))
    Tree[Now].Right <- tot + 1
    MakeTree(floor((a + b) / 2), b)
End

```

2.3 Chèn và xóa đoạn trong cây đoạn

Thêm trường Cover để ghi số lần một đoạn được phủ:

```

Type TreeNode = Record
  a, b, Left, Right, Cover: Longint;
End;

```

Khi xây cây, cần khởi tạo Cover bằng 0.

2.3.1 Chèn đoạn

Để chèn đoạn $[c, d]$, nếu đoạn của nút hiện tại nằm hoàn toàn trong $[c, d]$ thì tăng Cover; nếu không, tiếp tục đi xuống các con có giao với $[c, d]$.

```
Procedure Insert(Num)
Begin
  If (c < Tree[Num].a) and (Tree[Num].b < d) then
    Tree[Num].Cover <- Tree[Num].Cover + 1
  Else
    If c < floor((Tree[Num].a + Tree[Num].b) / 2) then
      Insert(Tree[Num].Left)
    If d > floor((Tree[Num].a + Tree[Num].b) / 2) then
      Insert(Tree[Num].Right)
End
```

2.3.2 Xóa đoạn

Để xóa một đoạn $[c, d]$ đã được chèn, thao tác đối xứng với chèn: khi gặp nút nằm trọn trong đoạn cần xóa, giảm Cover.

```
Procedure Delete(Num)
Begin
  If (c < Tree[Num].a) and (Tree[Num].b < d) then
    Tree[Num].Cover <- Tree[Num].Cover - 1
  Else
    If c < floor((Tree[Num].a + Tree[Num].b) / 2) then
      Delete(Tree[Num].Left)
    If d > floor((Tree[Num].a + Tree[Num].b) / 2) then
      Delete(Tree[Num].Right)
End
```

2.4 Ứng dụng cơ bản của cây đoạn

Sau khi nắm được ba thao tác xây, chèn và xóa, ta có thể giải những bài thống kê tĩnh cơ bản. Ví dụ: cho nhiều đoạn $[a, b]$ với $0 < a < b < 10000$ trên trục số, cần tính có bao nhiêu đoạn đơn vị $[k, k + 1]$ được phủ. Khi đọc mỗi đoạn $[a, b]$, gọi Insert(1). Sau khi chèn xong, duyệt cây để thống kê:

```
Procedure Count(Num)
Begin
  If Tree[Num].Cover > 0 then
    Number <- Number + (Tree[Num].b - Tree[Num].a)
  Else
    Count(Tree[Num].Left)
    Count(Tree[Num].Right)
End
```

Với các bài thống kê tĩnh đơn giản, cây đoạn rất thuận tiện. Khó khăn xuất hiện khi bài toán là thống kê động, chẳng hạn có thao tác xóa hoặc sửa một phần dữ liệu.

Ví dụ 1. Trên trục số thực hiện một dãy thao tác. Mỗi thao tác thuộc một trong hai loại: tô màu đoạn $[a, b]$, hoặc xóa màu trên đoạn $[a, b]$. Hỏi sau toàn bộ dãy thao tác, có bao nhiêu đoạn đơn vị $[k, k + 1]$ được tô màu.

Vấn đề nằm ở thao tác xóa. Trong cây đoạn cơ bản, ta chỉ xóa được chính đoạn đã từng chèn. Nếu chưa từng chèn đoạn $[3, 6]$ thì không thể trực tiếp xóa $[3, 6]$. Nhưng trong bài này, sau khi tô $[1, 15]$, ta có thể xóa màu trên $[4, 9]$; trong cây chỉ có đoạn $[1, 15]$, nên không thể xóa $[4, 9]$ theo cách cơ bản. Vì vậy cần cải tiến cây đoạn.

2.5 Cải tiến cây đoạn

Xét lại ví dụ trên. Sau khi tô $[1, 15]$ rồi xóa $[4, 9]$, đoạn $[1, 15]$ không còn tồn tại nguyên vẹn; phần còn lại là $[1, 4]$ và $[9, 15]$. Ta có thể nghĩ đến việc xóa $[1, 15]$ rồi chèn lại hai đoạn này. Nhưng nếu trước đó đã chèn cả $[8, 11]$ và $[1, 8]$, chỉ sửa $[1, 15]$ là chưa đủ, vì trạng thái của các đoạn con cũng không còn phản ánh đúng thực tế.

Trường hợp xấu hơn, nếu mọi nút trong cây gốc $[1, 15]$ đều từng được chèn, chẳng hạn các đoạn đơn vị, các đoạn dài hơn, rồi cuối cùng là $[1, 15]$, thì một thao tác xóa $[1, 15]$ khiến trạng thái của mọi nút sai. Khi đó phải sửa số nút tỷ lệ với kích thước cây. Với đoạn $[1, 30000]$, một thao tác xóa có thể cần gần 60000 lần sửa, còn kém hơn mô phỏng trực tiếp.

Để tránh điều này, ta thêm cho mỗi nút một trường đánh dấu, ký hiệu b_j . Ý tưởng là trì hoãn sửa các nút con cho đến khi thật sự cần truy cập chúng. Ví dụ, nếu toàn bộ cây gốc $[1, 5]$ đang được tô màu và ta xóa màu trên $[1, 5]$, thì mọi nút con đều sai về mặt logic. Nhưng nếu sau đó chỉ hỏi đoạn $[3, 5]$, các sửa đổi ở $[1, 2]$, $[2, 3]$, $[3, 4]$, $[4, 5]$ chưa chắc cần thực hiện ngay.

Cách làm cụ thể:

1. Sau khi xóa toàn bộ đoạn $[a, b]$, đánh dấu hai con của nó bằng $b_j = -1$.
2. Mỗi lần chuẩn bị truy cập một đoạn, trước hết kiểm tra nó có bị đánh dấu hay không. Nếu $b_j = -1$, đặt trạng thái của đoạn thành chưa phủ, xóa dấu của nó, rồi truyền dấu xóa xuống hai con.

Thủ tục cập nhật dấu:

```

Procedure Clear(Num)
Begin
  Tree[Num].Cover <- 0
  Tree[Num].bj <- 0
  Tree[Tree[Num].Left].bj <- -1
  Tree[Tree[Num].Right].bj <- -1
End

```

Dấu sẽ được truyền dọc theo đúng đường đi cần truy cập. Chẳng hạn muốn truy cập $[3, 4]$, ta phải đi qua cha $[3, 5]$; nếu cha bị đánh dấu, thao tác Clear sẽ truyền dấu xuống $[3, 4]$ và $[4, 5]$. Vì vậy khi thật sự đến $[3, 4]$, ta biết phải sửa trạng thái của nó. Nếu không bao giờ dùng tới $[3, 4]$, việc chưa sửa nó không ảnh hưởng đến đáp án. Nhờ đó, tính đúng đắn được giữ trong khi tránh được nhiều thao tác vô ích. Mỗi lần kiểm tra dấu và cập nhật tại một nút chỉ tốn $O(1)$.

Sau khi thêm trường đánh dấu, hai thao tác chính trong ví dụ 1 có thể mô tả như sau.

Thủ tục chèn Insert.

1. Nếu đoạn hiện tại bị đánh dấu, gọi Clear.

2. Nếu đoạn hiện tại đã được tô màu, thoát.
3. Nếu vùng tô phủ toàn bộ đoạn hiện tại, đặt đoạn hiện tại thành đã tô màu rồi thoát.
4. Nếu vùng tô giao với nửa trái, gọi Insert ở con trái.
5. Nếu vùng tô giao với nửa phải, gọi Insert ở con phải.

Thủ tục xóa Delete.

1. Nếu đoạn hiện tại đã bị đánh dấu, thoát, vì nó đã mang nghĩa vụ bị xóa.
2. Nếu vùng xóa phủ toàn bộ đoạn hiện tại, đặt đoạn hiện tại thành chưa tô, đánh dấu hai con rồi thoát.
3. Nếu đoạn hiện tại đang được tô, trước hết xóa cả đoạn và đánh dấu hai con. Sau đó chèn lại phần còn màu, tức $[a, c]$ nếu $a < c$ và $[d, b]$ nếu $d < b$.
4. Nếu đoạn hiện tại chưa được tô, tiếp tục gọi xóa ở các con có giao với vùng xóa.

Tư tưởng đánh dấu có thể tổng quát như sau. Khi một thao tác áp dụng lên toàn bộ đoạn $[a, b]$, ta có thể chỉ đánh dấu các con của $[a, b]$ mà chưa sửa toàn bộ cây con. Lý do là mọi nút bên dưới đều biểu diễn đoạn con của $[a, b]$, nên thay đổi ở $[a, b]$ cũng hàm ý thay đổi ở chúng. Hình thức của dấu không cố định; mỗi bài cần một cách thiết kế dấu phù hợp.

Ví dụ 2: đặt chỗ tàu hỏa. Bộ đường sắt Byteotia xây dựng một tuyến gồm n thành phố đánh số từ 1 đến n . Mỗi chuyến tàu có m ghế, và trên bất kỳ đoạn giữa hai ga liên tiếp nào cũng không được chở quá m hành khách. Hệ thống nhận tuần tự các yêu cầu đặt chỗ; mỗi yêu cầu có dạng (k_1, k_2, v) , nghĩa là cần v ghế từ ga k_1 đến ga k_2 . Nếu trên toàn bộ đoạn yêu cầu còn đủ ghế thì chấp nhận, ngược lại từ chối. Không cho phép chấp nhận một phần yêu cầu. Với mỗi yêu cầu, cần in T nếu chấp nhận và N nếu từ chối.

Giới hạn: $1 \leq n, m, r \leq 60000, 1 \leq k_1 < k_2 \leq n, 1 \leq v \leq m$.

Bài này cần hỏi trạng thái một đoạn, rồi nếu yêu cầu được chấp nhận thì sửa số ghế trên đoạn đó. Ta tiếp tục dùng đánh dấu lười. Vì trọng tâm là truy vấn “còn đủ ghế hay không”, mỗi nút lưu Seat, là số ghế còn lại nhỏ nhất trên đoạn của nút. Khi trạng thái của con trái hoặc con phải thay đổi, cập nhật:

$$[a, b].\text{Seat} = \min(\text{Seat}_{\text{con trái}}, \text{Seat}_{\text{con phải}}).$$

Ban đầu, mọi nút có $\text{Seat} = m$.

Nếu cần giảm toàn bộ đoạn của một nút đi v ghế, ta giảm Seat của nút đó và cộng dấu bj của hai con thêm giá trị âm tương ứng. Trước khi truy cập một nút, nếu bj khác 0, gọi:

Procedure Clear(Num)

Begin

Tree[Num].Seat <- Tree[Num].Seat + Tree[Num].bj

Tree[Tree[Num].Left].bj <- Tree[Tree[Num].Left].bj + Tree[Num].bj

Tree[Tree[Num].Right].bj <- Tree[Tree[Num].Right].bj + Tree[Num].bj

Tree[Num].bj <- 0

End

Hàm Can kiểm tra yêu cầu:

1. Nếu nút có dấu, gọi Clear.
2. Nếu đoạn yêu cầu phủ toàn bộ nút, trả về liệu Seat có ít nhất là v hay không.

3. Nếu đoạn yêu cầu cắt qua trung điểm, kết quả là phép and của hai con.
4. Nếu đoạn yêu cầu nằm hẳn một phía, đi xuống con tương ứng.

Thủ tục sửa ghế sau khi yêu cầu được chấp nhận:

1. Nếu nút có dấu, gọi Clear.
2. Nếu đoạn yêu cầu phủ toàn bộ nút, giảm Seat của nút đi v , truyền dấu giảm xuống hai con rồi thoát.
3. Với con có giao với đoạn yêu cầu, gọi đệ quy; với con không giao, vẫn gọi Clear nếu nó đang mang dấu, vì bước cập nhật ở nút cha cần giá trị Seat đúng của cả hai con.
4. Đặt Seat của nút hiện tại bằng giá trị nhỏ hơn của hai con.

2.6 Mở rộng cây đoạn

Cây đoạn một chiều xử lý các bài thống kê tuyến tính. Trong thực tế, ta còn gặp thống kê trên mặt phẳng hoặc trong không gian. Vì vậy cần mở rộng cây đoạn thành **cây hình chữ nhật** cho bài toán hai chiều và **cây hình hộp** cho bài toán ba chiều.

Có hai cách chuyển cây đoạn một chiều thành hai chiều. Cách thứ nhất là tại mỗi nút của cây đoạn chính lại gắn một cây đoạn phụ. Nói cách khác, đó là “cây trong cây”. Ví dụ, ở nút $[1, 3]$ của cây chính, đoạn $[3, 5]$ trong cây phụ biểu diễn hình chữ nhật $(1, 3, 3, 5)$, trong đó (x_1, y_1, x_2, y_2) ký hiệu hình chữ nhật có góc trái dưới (x_1, y_1) và góc phải trên (x_2, y_2) .

Với cách này, hình chữ nhật (x_1, y_1, x_2, y_2) cần không gian

$$O((2(x_2 - x_1) - 1)(2(y_2 - y_1) - 1)) = O(Long_x Long_y),$$

trong đó $Long_x, Long_y$ là chiều dài và chiều rộng. Nếu có n thao tác, thời gian là

$$O(n \log_2 Long_x \log_2 Long_y).$$

Nhược điểm là phải xử lý hai tầng cây nên cài đặt phức tạp hơn.

Cách thứ hai là để mỗi nút trực tiếp biểu diễn một hình chữ nhật và chia nó thành bốn hình chữ nhật con. Với hình chữ nhật (x_1, y_1, x_2, y_2) , đặt

$$x_m = \left\lfloor \frac{x_1 + x_2}{2} \right\rfloor, \quad y_m = \left\lfloor \frac{y_1 + y_2}{2} \right\rfloor.$$

Bốn con lần lượt là:

$$(x_1, y_1, x_m, y_m), \quad (x_m, y_1, x_2, y_m), \\ (x_1, y_m, x_m, y_2), \quad (x_m, y_m, x_2, y_2).$$

Đây là tư tưởng chia tư. Không gian vẫn là $O(Long_x Long_y)$, nhưng chỉ có một tầng nên dễ thao tác hơn và dấu lười vẫn áp dụng tự nhiên. Tuy vậy thời gian trong trường hợp xấu có thể lên tới $O(n Long_x)$, kém hơn cách cây trong cây.

Với bài toán nhiều chiều, cách cây trong cây gần như không khả thi. Ta có thể bắt chước cách thứ hai. Với không gian n chiều, cây có gốc

$$(a_1, a_2, \dots, a_n, b_1, b_2, \dots, b_n),$$

trong đó $(a_1, \dots, a_n)$ là góc thấp và $(b_1, \dots, b_n)$ là góc cao. Khi xây cây, ta không chia đôi hay chia tư nữa mà chia thành 2^n phần. Số nút tỷ lệ với

$$2^n (b_1 - a_1)(b_2 - a_2) \cdots (b_n - a_n).$$

2.7 Tổng kết về cây đoạn

Sau khi được cải tiến và mở rộng, cây đoạn là một công cụ mạnh cho các bài toán thống kê. Nó có phạm vi áp dụng rộng, tốc độ tốt và cài đặt thuận tiện. Tuy nhiên cây đoạn vẫn có nhược điểm, nhất là về bộ nhớ khi miền tọa độ lớn hoặc số chiều tăng; phần sau sẽ so sánh điểm này với cấu trúc chữ nhật.

3 Cấu trúc chữ nhật

Cấu trúc chữ nhật là phương pháp xử lý thống kê các hình chữ nhật trên mặt phẳng. Nhiều bài toán thống kê, sau khi mô hình hóa toán học, có thể chuyển thành bài toán cấu trúc chữ nhật. Nguyên mẫu của phương pháp này là cây đoạn.

3.1 Cây đoạn

Xét lại ví dụ 1 nhưng thay đổi điều kiện: thao tác tô màu có thể dùng nhiều màu, và màu tô sau phủ màu tô trước trên cùng một đoạn. Với mỗi màu, cần tính số đoạn đơn vị có màu đó.

Nếu các đoạn trong tập hiện tại đôi một không chồng lấn, ta chỉ cần cộng độ dài những đoạn có cùng màu. Vì trên thực tế các đoạn có thể chồng lấn, ta dùng cây đoạn để duy trì tập đoạn sao cho các đoạn trong tập luôn đôi một không giao nhau. Khi chèn đoạn mới $[c, d]$, xét từng đoạn cũ $[a, b]$. Nếu $a \geq d$ hoặc $c \leq b$ thì hai đoạn không chồng lấn; ngược lại chúng chồng lấn. Đặt giao của chúng là $[k_1, k_2]$. Khi đó:

- nếu $a < k_1$, thêm đoạn $[a, k_1]$;
- nếu $k_2 < b$, thêm đoạn $[k_2, b]$;
- xóa đoạn cũ $[a, b]$.

Sau khi mọi đoạn đã được chèn và xử lý, tập đoạn không còn chồng lấn, nên việc thống kê theo màu chỉ còn là cộng độ dài.

3.1.1 Cấu trúc dữ liệu của đoạn

```
Type Segment = Record  
  a, b: Longint;  
End;
```

Thông thường ta thêm các trường mô tả trạng thái, chẳng hạn Colour để ghi màu của đoạn.

3.1.2 Hàm kiểm tra giao nhau

```
Function Cross(a, b, c, d)  
Begin  
  If (a >= d) or (c >= b) then Cross <- false  
  Else Cross <- true  
End
```

3.1.3 Thủ tục cắt đoạn

```
Procedure Cut(Num, c, d)
```

```

Begin
  If Line[Num].a < c then Add(Line[Num].a, c)
  If d < Line[Num].b then Add(d, Line[Num].b)
  Delete(Num)
End

```

Trong đó Add thêm một đoạn vào tập:

```

Procedure Add(a, b)
Begin
  tot <- tot + 1
  Line[tot].a <- a
  Line[tot].b <- b
End

```

Thủ tục Delete có thể xóa bằng cách đưa đoạn cuối tập vào vị trí cần xóa:

```

Procedure Delete(Num)
Begin
  Line[Num] <- Line[tot]
  tot <- tot - 1
End

```

3.2 Cắt hình chữ nhật

Tương tự, giả sử tập hiện tại có hình chữ nhật (x_1, y_1, x_2, y_2) và ta chèn hình chữ nhật mới (x_3, y_3, x_4, y_4) . Quan hệ vị trí giữa hai hình chữ nhật có nhiều trường hợp, nhưng nếu tách quá trình theo hai trục thì thao tác trở nên rõ ràng.

Trước hết cắt theo hướng x . Lấy giao của hai đoạn (x_1, x_2) và (x_3, x_4) là (k_1, k_2) . Phần nằm ngoài giao theo hướng x tạo ra tối đa hai hình chữ nhật:

$$(x_1, y_1, k_1, y_2), \quad (k_2, y_1, x_2, y_2).$$

Sau đó chỉ cần tiếp tục cắt phần còn lại (k_1, y_1, k_2, y_2) theo hướng y . Nếu giao của (y_1, y_2) và (y_3, y_4) là (k_3, k_4) , ta thêm tối đa hai hình chữ nhật:

$$(k_1, y_1, k_2, k_3), \quad (k_1, k_4, k_2, y_2).$$

Phần còn lại bị hình chữ nhật mới phủ, nên xóa hình chữ nhật cũ.

Thủ tục cắt có thể viết:

```

Procedure Cut(x1, y1, x2, y2, Direction)
Var k1, k2
Begin
  Case Direction of
    1: Begin
      k1 <- Max(x1, x3)
      k2 <- Min(x2, x4)
      If x1 < k1 then Add(x1, y1, k1, y2)
      If k2 < x2 then Add(k2, y1, x2, y2)

```



```

        Cut(k1, y1, k2, y2, Direction + 1)
    End
2: Begin
    k1 <- Max(y1, y3)
    k2 <- Min(y2, y4)
    If y1 < k1 then Add(x1, y1, x2, k1)
    If k2 < y2 then Add(x1, k2, x2, y2)
End
End
End

```

3.3 Mở rộng cắt hình chữ nhật

Từ cắt hình chữ nhật, ta mở rộng được sang cắt hình hộp, thậm chí cắt trong không gian n chiều. Hai vật thể n chiều có phần chồng lấn khi và chỉ khi hình chiếu của chúng trên cả n trục đều có giao. Cách cắt cũng tương tự: cắt theo trục x , rồi theo trục y , tiếp đến trục z , và cứ thế cho đến trục thứ n .

Khi số chiều tăng, nếu phân nhánh thủ công theo từng trục thì chương trình rất dài. Ta có thể biểu diễn một vật thể n chiều bằng hai mảng

$$(a[1], a[2], \dots, a[n]), \quad (b[1], b[2], \dots, b[n]),$$

rồi thay thủ tục Add và Cut bằng vòng lặp qua các chiều. Như vậy mã ngắn hơn nhiều mà vẫn giữ nguyên ý tưởng.

Ví dụ 3: phủ vệ tinh. Một vệ tinh phủ một hình lập phương trong hệ tọa độ không gian. Vệ tinh nằm ở tâm hình lập phương; trạng thái của nó được mô tả bởi bộ (x, y, z, r) , trong đó (x, y, z) là tâm và r là nửa cạnh. Các cạnh song song với trục tọa độ, nên thể tích phủ bởi một vệ tinh là

$$V = (2r)^3 = 8r^3.$$

Có N vệ tinh, $1 \leq N \leq 100$, với $-1000 \leq x, y, z \leq 1000$ và $1 \leq r \leq 200$. Cần tính thể tích hợp của các hình lập phương.

Bài này dùng cắt hình hộp. Mỗi khi đọc một hình hộp mới $(x_3, y_3, z_3, x_4, y_4, z_4)$, xét nó với từng hình hộp cũ $(x_1, y_1, z_1, x_2, y_2, z_2)$. Nếu có giao trên cả ba trục, cắt hình hộp cũ thành các phần không bị hình hộp mới phủ. Sau khi xử lý xong toàn bộ dữ liệu, cộng thể tích mọi hình hộp còn lại để được đáp án.

Cần chú ý rằng các hình hộp mới sinh ra khi cắt một hình hộp cũ sẽ không chồng lấn với hình hộp vừa đọc. Vì vậy, trước khi xử lý hình hộp mới, ghi `tot1 <- tot`; vòng kiểm tra giao chỉ cần chạy đến `tot1`. Các hình hộp sinh ra sau vị trí này không cần kiểm tra lại với hình hộp mới, nhờ đó tiết kiệm thời gian.

3.4 Ứng dụng của cắt hình chữ nhật

Nếu cắt hình chữ nhật chỉ dùng cho bài toán hình học thuần túy thì phạm vi không rộng. Giá trị của nó nằm ở chỗ nhiều bài thống kê có thể được mô hình hóa thành các hình chữ nhật.

Ví dụ 4: hệ thống thống kê chiến trường. Năm 2050, chiến tranh giữa loài người và người ngoài hành tinh trở nên ác liệt. Loài người phát minh một siêu vũ khí có thể tấn công nhiều vị trí kề nhau. Mọi người ngoài hành tinh có phòng thủ không vượt quá sức công phá của vũ khí sẽ bị tiêu diệt. Hệ thống cần xử lý hai loại thông tin:

1. Người ngoài hành tinh tăng viện một đơn vị có phòng thủ v tại mỗi vị trí trong đoạn $[x_1, x_2]$.
2. Loài người dùng vũ khí có sức công phá v tấn công mọi vị trí trong đoạn $[x_1, x_2]$; hệ thống phải trả về số người ngoài hành tinh bị tiêu diệt.

Ghi chú: một đơn vị có phòng thủ i gồm i người, trong đó người thứ j có phòng thủ j .

Ở phiên bản dùng trong phần ứng dụng này, giới hạn là $0 < n \leq 30000$, $0 < x_1 \leq x_2 \leq n$, $0 < v \leq 30000$, $0 < m \leq 2000$.

Ban đầu bài này trông không giống cắt hình chữ nhật. Nhưng nếu dựng hệ tọa độ với trục hoành là vị trí và trục tung là mức phòng thủ, thao tác thêm một đơn vị phòng thủ v trên $[x_1, x_2]$ tương đương với thêm hình chữ nhật

$$(x_1 - 1, 0, x_2, v).$$

Lý do là đơn vị phòng thủ v chứa các cá thể có mức phòng thủ $1, 2, \dots, v$. Tương tự, tấn công trên $[x_1, x_2]$ với sức công phá v tương đương với xóa phần giao với hình chữ nhật $(x_1 - 1, 0, x_2, v)$.

Khác với bài tô màu, khi thêm quân ta không cần cắt các hình chữ nhật cũ, vì nhiều đơn vị có thể cùng đứng ở một ô. Chỉ khi dùng vũ khí, ta cắt các hình chữ nhật giao với vùng tấn công và đồng thời cộng diện tích phần giao vào số người bị tiêu diệt.

Qua ví dụ này có thể thấy cắt hình chữ nhật không chỉ dành cho bài hình học. Khi mô hình hóa được bài toán thống kê thành các hình chữ nhật chồng lấn, đây cũng là một phương pháp đáng dùng.

4 So sánh cây đoạn và cắt hình chữ nhật

Cây đoạn và cắt hình chữ nhật đều là công cụ xử lý thống kê động, nhưng chúng khác nhau đáng kể. Để chọn đúng công cụ, trước hết cần so sánh độ phức tạp.

4.1 Độ phức tạp của cây đoạn

4.1.1 Bộ nhớ

1. Cây đoạn một chiều cần $O(Long_x)$ bộ nhớ, trong đó $Long_x$ là độ dài đoạn lớn nhất.
2. Cây đoạn hai chiều cần $O(Long_x Long_y)$ bộ nhớ, trong đó $Long_x, Long_y$ là chiều dài và chiều rộng lớn nhất.
3. Cây đoạn ba chiều cần $O(Long_x Long_y Long_z)$ bộ nhớ.

4.1.2 Thời gian

Với n thao tác:

1. Cây đoạn một chiều có thời gian $O(n \log_2 Long_x)$.
2. Cây đoạn hai chiều có thời gian $O(n \log_2 Long_x \log_2 Long_y)$.
3. Cây đoạn ba chiều có thời gian $O(n \log_2 Long_x \log_2 Long_y \log_2 Long_z)$.

4.2 Độ phức tạp của cắt hình chữ nhật

Mỗi khi đọc một hình chữ nhật mới, ta phải so sánh nó với các hình chữ nhật trong tập hiện tại để xem có chồng lấn hay không. Vì vậy thời gian có dạng $O(mn)$, trong đó m là số hình chữ nhật của dữ liệu vào,

còn n là số hình chữ nhật lớn nhất từng xuất hiện trong tập trong quá trình xử lý. Việc ước lượng n chính là vấn đề bộ nhớ.

4.2.1 Bộ nhớ

Bộ nhớ của cấu hình chữ nhật phụ thuộc vào đỉnh n : tập có thể có nhiều nhất bao nhiêu hình chữ nhật thì cần mảng lớn bấy nhiêu. Đại lượng này khó tính vì phụ thuộc mạnh vào cách các hình chữ nhật được đặt. Tác giả thử sinh ngẫu nhiên các hình chữ nhật trong miền $0 \leq x_1 < x_2 \leq 60000$, $0 \leq y_1 < y_2 \leq 60000$. Với mỗi giá trị m , sinh 10 bộ dữ liệu và lấy trung bình đỉnh n :

số hình chữ nhật m	100	500	1000	5000	10000	50000	100000
đỉnh n	239	479	680	1015	1296	1741	2092

Khi m nhỏ, chẳng hạn $m = 100$, đỉnh $n = 239$ lớn hơn hai lần m . Nhưng khi m tăng nhanh, n tăng khá chậm: với $m = 5000$, $n = 1015$; với $m = 10000$, $n = 1296$. Điều này cho thấy với dữ liệu ngẫu nhiên, số hình chữ nhật trong tập không tỷ lệ tuyến tính với số hình chữ nhật đầu vào, và bộ nhớ thường khá nhỏ.

Tuy nhiên đó chỉ là dữ liệu ngẫu nhiên. Có thể dựng dữ liệu khiến cấu hình chữ nhật rất tệ. Giả sử có m hình chữ nhật. Trước hết đặt k hình chữ nhật lồng nhau: hình đầu là $(1, 1, x, y)$ với x, y rất lớn, hình sau thu nhỏ mỗi biên một đơn vị so với hình trước, tức nếu hình trước là (x_1, y_1, x_2, y_2) thì hình sau là $(x_1 + 1, y_1 + 1, x_2 - 1, y_2 - 1)$. Hình thứ $t + 1$ sẽ cắt hình thứ t thành bốn mảnh, nên sau k hình có $4(k - 1) + 1 = 4k - 3$ hình chữ nhật.

Sau đó đặt $m - k$ hình chữ nhật dạng

$$(k + 1, 0, k + 2, y + 1), \quad (k + 3, 0, k + 4, y + 1), \quad \dots$$

Mỗi hình như vậy giao với $2k - 1$ hình trong tập, nên sau khi cắt sẽ sinh thêm $2k - 1$ hình. Tổng số hình chữ nhật có thể đạt

$$(m - k)(2k - 1) + 4k - 3 = -2k^2 + (2m + 5)k - (m + 3).$$

Biểu thức bậc hai này đạt cực đại khi $k = (2m + 5)/4$, với giá trị cỡ $m^2/2 + 3m/2 + 1/8$. Vì vậy trong dữ liệu được dựng theo điểm yếu của thuật toán, bộ nhớ có thể đạt $O(m^2)$.

4.2.2 Thời gian

Khi biết đỉnh n , thời gian là $O(mn)$. Với dữ liệu ngẫu nhiên ở bảng trên, khi $m = 10000$ và $n = 1296$ thì vẫn có thể chấp nhận; khi $m = 50000$ và $n = 1741$ thì đã khá nặng. Với dữ liệu cực đoan, vì n có thể là $O(m^2)$, thời gian trở thành $O(m^3)$; chỉ khoảng 500 hình chữ nhật cũng có thể dẫn đến hơn một trăm triệu phép toán.

4.3 Phạm vi thích hợp

Từ độ phức tạp có thể rút ra cách chọn phương pháp.

Cây đoạn có bộ nhớ cố định theo kích thước miền, chẳng hạn $O(\text{Long}_x)$ trong một chiều. Nếu miền tọa độ rất lớn, bộ nhớ của cây đoạn sẽ lớn đến mức không chịu nổi, đặc biệt ở cây hình chữ nhật và cây hình hộp. Ví dụ, nếu hình hộp có chiều dài, rộng, cao đều dưới 1000, bộ nhớ đã có thể lên tới $8 \cdot 10^9$ đơn vị, không khả thi.

Cắt hình chữ nhật có ưu thế trong trường hợp miền rất lớn. Một hình chữ nhật chỉ cần bốn trường để lưu, một hình hộp chỉ cần sáu trường, không phụ thuộc vào biên $\text{Long}_x, \text{Long}_y$ hay các chiều khác. Vì

vậy các giới hạn như $-1000 \leq x, y, z \leq 1000$ trong bài vệ tinh, hoặc $0 < n \leq 30000$, $0 < x_1 \leq x_2 \leq n$, $0 < v \leq 30000$ trong bài chiến trường, khiến cây đoạn nhiều chiều khó dùng, nhưng cấu hình chữ nhật vẫn phù hợp nếu số thao tác không lớn.

Ngược lại, cây đoạn có thời gian rất tốt, chỉ $O(n \log_2 Long_x)$ trong một chiều, nên khi số thao tác lớn và miền không quá rộng, cây đoạn là lựa chọn mạnh. Cấu hình chữ nhật suy giảm nhanh khi số thao tác lớn. Trong bài vệ tinh, số hình hộp tối đa chỉ là 100, nên cấu hình hộp là lựa chọn tự nhiên.

Về độ phức tạp lập trình, cả hai phương pháp đều tương đối dễ cài đặt sau khi nắm đúng mô hình. Kết luận thực dụng là: với bài có biên nhỏ và nhiều thao tác, chọn cây đoạn; với bài có biên lớn và ít thao tác, chọn cấu hình chữ nhật.

5 Kết luận

Qua việc xem xét tư tưởng, thao tác cơ bản, cải tiến và mở rộng của cây đoạn cũng như cấu hình chữ nhật, ta có được cái nhìn rõ hơn về hai công cụ xử lý thống kê động này. So sánh độ phức tạp, ưu khuyết điểm và phạm vi áp dụng giúp ta tích lũy kinh nghiệm chọn phương pháp phù hợp cho từng bài.

Bài viết chỉ giới thiệu hai phương pháp và phạm vi vẫn còn hạn chế. Tuy vậy, điều quan trọng hơn là năng lực phát hiện vấn đề, đặt câu hỏi, suy nghĩ và giải quyết vấn đề. Nếu việc giới thiệu hai phương pháp này đồng thời gợi mở thêm sự quan tâm đến các bài toán thống kê và các dạng bài khác, thì mục tiêu của bài viết đã đạt được.

Tài liệu tham khảo

1. Đề thi NOI 1997.
2. Đề thi trực tuyến OIBH.

Phụ lục

Các chương trình trong phụ lục gốc là Pascal. Dưới đây là bản dựng lại theo đúng thuật toán và các thủ tục chính; tên biến được giữ gần với bản gốc để dễ đối chiếu.

Phụ lục 1: ví dụ 1, tô và xóa màu trên trục số

```
Const Maxn = 120000;
Type TreeNode = Record
    a, b, Left, Right: Longint;
    Cover, bj: Shortint;
End;

Procedure MakeTree(a, b: Longint);
Var Now: Longint;
Begin
    Inc(tot);
    Now := tot;
    Tree[Now].a := a;
    Tree[Now].b := b;
```

```

    If a + 1 < b then Begin
        Tree[Now].Left := tot + 1;
        MakeTree(a, (a + b) shr 1);
        Tree[Now].Right := tot + 1;
        MakeTree((a + b) shr 1, b);
    End;
End;

Procedure Clean(Num: Longint);
Begin
    Tree[Num].Cover := 0;
    Tree[Num].bj := 0;
    Tree[Tree[Num].Left].bj := -1;
    Tree[Tree[Num].Right].bj := -1;
End;

Procedure Insert(Num, c, d: Longint);
Var Mid: Longint;
Begin
    If Tree[Num].bj = -1 then Clean(Num);
    If Tree[Num].Cover = 1 then Exit;
    If (c <= Tree[Num].a) and (d >= Tree[Num].b) then Begin
        Tree[Num].Cover := 1;
        Exit;
    End;
    Mid := (Tree[Num].a + Tree[Num].b) shr 1;
    If c < Mid then Insert(Tree[Num].Left, c, d);
    If d > Mid then Insert(Tree[Num].Right, c, d);
End;

Procedure Delete(Num, c, d: Longint);
Var Mid: Longint;
Begin
    If Tree[Num].bj = -1 then Exit;
    If (c <= Tree[Num].a) and (d >= Tree[Num].b) then Begin
        Tree[Num].Cover := 0;
        Tree[Tree[Num].Left].bj := -1;
        Tree[Tree[Num].Right].bj := -1;
        Exit;
    End;
    If Tree[Num].Cover = 1 then Begin
        Tree[Num].Cover := 0;
        Tree[Tree[Num].Left].bj := -1;
        Tree[Tree[Num].Right].bj := -1;
        If Tree[Num].a < c then Insert(Num, Tree[Num].a, c);
        If d < Tree[Num].b then Insert(Num, d, Tree[Num].b);
    End
    Else Begin

```

```

    Mid := (Tree[Num].a + Tree[Num].b) shr 1;
    If c < Mid then Delete(Tree[Num].Left, c, d);
    If d > Mid then Delete(Tree[Num].Right, c, d);
  End;
End;

```

```

Procedure Calculate(Num: Longint);
Begin
  If Num = 0 then Exit;
  If Tree[Num].bj = -1 then Exit;
  If Tree[Num].Cover = 1 then Begin
    Inc(Ans, Tree[Num].b - Tree[Num].a);
    Exit;
  End;
  Calculate(Tree[Num].Left);
  Calculate(Tree[Num].Right);
End;

```

Phụ lục 2: ví dụ 2, đặt chỗ tàu hỏa

```

Type TreeNode = Record
  Seat, bj, Left, Right, a, b: Longint;
End;

```

```

Procedure MakeTree(a, b: Longint);
Var Now: Longint;
Begin
  Inc(tot);
  Now := tot;
  Tree[Now].a := a;
  Tree[Now].b := b;
  Tree[Now].Seat := m;
  If a + 1 < b then Begin
    Tree[Now].Left := tot + 1;
    MakeTree(a, (a + b) shr 1);
    Tree[Now].Right := tot + 1;
    MakeTree((a + b) shr 1, b);
  End;
End;

```

```

Procedure Clear(Num: Longint);
Begin
  If Tree[Num].bj <> 0 then Begin
    Inc(Tree[Num].Seat, Tree[Num].bj);
    Inc(Tree[Tree[Num].Left].bj, Tree[Num].bj);
    Inc(Tree[Tree[Num].Right].bj, Tree[Num].bj);
    Tree[Num].bj := 0;
  End;
End;

```

```

End;

Function Can(Num, c, d, v: Longint): Boolean;
Var Mid: Longint;
Begin
  Clear(Num);
  If (c <= Tree[Num].a) and (Tree[Num].b <= d) then Begin
    Can := (Tree[Num].Seat >= v);
    Exit;
  End;
  Mid := (Tree[Num].a + Tree[Num].b) shr 1;
  If (c < Mid) and (d > Mid) then
    Can := Can(Tree[Num].Left, c, d, v) and
           Can(Tree[Num].Right, c, d, v)
  Else if c < Mid then
    Can := Can(Tree[Num].Left, c, d, v)
  Else
    Can := Can(Tree[Num].Right, c, d, v);
End;

Procedure Delete(Num, c, d, v: Longint);
Var Mid: Longint;
Begin
  Clear(Num);
  If (c <= Tree[Num].a) and (Tree[Num].b <= d) then Begin
    Dec(Tree[Num].Seat, v);
    Dec(Tree[Tree[Num].Left].bj, v);
    Dec(Tree[Tree[Num].Right].bj, v);
    Exit;
  End;
  Mid := (Tree[Num].a + Tree[Num].b) shr 1;
  If c < Mid then Delete(Tree[Num].Left, c, d, v)
  Else Clear(Tree[Num].Left);
  If Mid < d then Delete(Tree[Num].Right, c, d, v)
  Else Clear(Tree[Num].Right);
  If Tree[Tree[Num].Left].Seat < Tree[Tree[Num].Right].Seat then
    Tree[Num].Seat := Tree[Tree[Num].Left].Seat
  Else
    Tree[Num].Seat := Tree[Tree[Num].Right].Seat;
End;

```

Phụ lục 3: ví dụ 3, phủ vệ tính

```

Type Block = Record
  x1, y1, z1, x2, y2, z2: Longint;
End;

Function Cross(x1, x2, x3, x4: Longint): Boolean;

```

```

Begin
  Cross := not ((x1 >= x4) or (x3 >= x2));
End;

Procedure Add(x1, y1, z1, x2, y2, z2: Longint);
Begin
  Inc(tot);
  Cubic[tot].x1 := x1; Cubic[tot].y1 := y1; Cubic[tot].z1 := z1;
  Cubic[tot].x2 := x2; Cubic[tot].y2 := y2; Cubic[tot].z2 := z2;
End;

Procedure Cut(x1, y1, z1, x2, y2, z2, Direction: Longint);
Var k1, k2: Longint;
Begin
  Case Direction of
    1: Begin
      k1 := Max(x1, Now.x1); k2 := Min(x2, Now.x2);
      If x1 < k1 then Add(x1, y1, z1, k1, y2, z2);
      If k2 < x2 then Add(k2, y1, z1, x2, y2, z2);
      Cut(k1, y1, z1, k2, y2, z2, Direction + 1);
    End;
    2: Begin
      k1 := Max(y1, Now.y1); k2 := Min(y2, Now.y2);
      If y1 < k1 then Add(x1, y1, z1, x2, k1, z2);
      If k2 < y2 then Add(x1, k2, z1, x2, y2, z2);
      Cut(x1, k1, z1, x2, k2, z2, Direction + 1);
    End;
    3: Begin
      k1 := Max(z1, Now.z1); k2 := Min(z2, Now.z2);
      If z1 < k1 then Add(x1, y1, z1, x2, y2, k1);
      If k2 < z2 then Add(x1, y1, k2, x2, y2, z2);
    End;
  End;
End;

Procedure Main;
Var i, j, tot1: Longint;
Begin
  For i := 1 to n do Begin
    Readln(x, y, z, r);
    Now.x1 := x - r; Now.y1 := y - r; Now.z1 := z - r;
    Now.x2 := x + r; Now.y2 := y + r; Now.z2 := z + r;
    j := 0; tot1 := tot;
    While j < tot1 do Begin
      Inc(j);
      If Cross(Cubic[j].x1, Cubic[j].x2, Now.x1, Now.x2) and
        Cross(Cubic[j].y1, Cubic[j].y2, Now.y1, Now.y2) and
        Cross(Cubic[j].z1, Cubic[j].z2, Now.z1, Now.z2) then Begin

```



```

        Cut(Cubic[j].x1, Cubic[j].y1, Cubic[j].z1,
            Cubic[j].x2, Cubic[j].y2, Cubic[j].z2, 1);
        Cubic[j] := Cubic[tot1];
        Cubic[tot1] := Cubic[tot];
        Dec(tot); Dec(tot1); Dec(j);
    End;
End;
Add(Now.x1, Now.y1, Now.z1, Now.x2, Now.y2, Now.z2);
End;
End;

```

Phụ lục 4: ví dụ 4, hệ thống thống kê chiến trường

```

Type Rectangle = Record
    x1, y1, x2, y2: Longint;
End;

Procedure Add(x1, y1, x2, y2: Longint);
Begin
    Inc(tot);
    Rect[tot].x1 := x1; Rect[tot].y1 := y1;
    Rect[tot].x2 := x2; Rect[tot].y2 := y2;
End;

Function Cross(x1, x2, x3, x4: Longint): Boolean;
Begin
    Cross := not ((x1 >= x4) or (x3 >= x2));
End;

Procedure Cut(x1, y1, x2, y2, Direction: Longint);
Var k1, k2: Longint;
Begin
    Case Direction of
        1: Begin
            k1 := Max(x1, Now.x1); k2 := Min(x2, Now.x2);
            If x1 < k1 then Add(x1, y1, k1, y2);
            If k2 < x2 then Add(k2, y1, x2, y2);
            Cut(k1, y1, k2, y2, Direction + 1);
        End;
        2: Begin
            k1 := Max(y1, Now.y1); k2 := Min(y2, Now.y2);
            If y1 < k1 then Add(x1, y1, x2, k1);
            If k2 < y2 then Add(x1, k2, x2, y2);
            Inc(Kill_Num, (x2 - x1) * (k2 - k1));
        End;
    End;
End;
End;

```

```

Procedure Add_Army;
Begin
    Add(Now.x1, Now.y1, Now.x2, Now.y2);
End;

Procedure Kill_Army;
Var i, tot1: Longint;
Begin
    Kill_Num := 0;
    tot1 := tot;
    i := 0;
    While i < tot1 do Begin
        Inc(i);
        If Cross(Rect[i].x1, Rect[i].x2, Now.x1, Now.x2) and
            Cross(Rect[i].y1, Rect[i].y2, Now.y1, Now.y2) then Begin
            Cut(Rect[i].x1, Rect[i].y1, Rect[i].x2, Rect[i].y2, 1);
            Rect[i] := Rect[tot1];
            Rect[tot1] := Rect[tot];
            Dec(tot); Dec(tot1); Dec(i);
        End;
    End;
    Writeln(Kill_Num);
End;

Procedure Main;
Begin
    For i := 1 to m do Begin
        Readln(k, x1, x2, v);
        Now.x1 := x1 - 1; Now.y1 := 0;
        Now.x2 := x2;      Now.y2 := v;
        If k = 1 then Add_Army
        Else Kill_Army;
    End;
End;

```

Phụ lục 5: nguyên mẫu bài hệ thống thống kê chiến trường

Phiên bản gốc của bài có cùng mô tả thao tác như ví dụ 4, nhưng giới hạn nhỏ hơn:

$$0 < n \leq 1000, \quad 0 < x_1 \leq x_2 \leq n, \quad 0 < v \leq 1000, \quad 0 < m \leq 2000.$$

Dữ liệu vào gồm n, m , sau đó là m dòng k, x_1, x_2, v với $k = 1$ hoặc $k = 2$. Chương trình in lần lượt các kết quả cần trả về của những thao tác tấn công.

Ví dụ:

dữ liệu vào	kết quả
3 5	
1 1 3 4	
2 1 2 3	6
1 1 2 2	9
1 2 3 1	
2 2 3 5	